STRAPI With NEXT.Js

1.INTRODUCTION:

Strapi:

- Strapi is a free, open-source, and self-hosted headless CMS built on Node.js. It separates the content management backend from the presentation layer, allowing developers to use any frontend technology to build websites, mobile apps, and other applications that pull content via APIs.
- Strapi provides an admin panel for content creators to manage data and uses APIs (REST or GraphQL) to deliver that content to any device.

How to connect with NextJs

- Create a next Js project usng npx create-strapi-app@latest backend.

Set up a configuration steps

After run npx create-strapi-app@latest backend,

It will run the terminal follow as

? Do you want to use the default database (sqlite) ? Yes // If you want use another database put no it will show other option configurations on terminal.
? Start with an example structure & data? No
? Start with Typescript? Yes
? Install dependencies with npm?
Yes
? Initialize a git repository? Yes

After complete it will create a project and run the terminal as
npm run develop
to see the localhost run on **http://localhost:1338**
to go strapi admin page to login / sign up to use

2. Main Concepts of Starpi

- Admin Panel
- Content Type builder
- Content Manager

2.1 Admin Panel :

- After configuration done it will go to Admin Panel,it like a DashBoard page.
- The admin panel acts as Strapi's back office for managing content types, entries, and both administrator and end-user accounts.

2.2 Content Type builder:

The Content-type Builder is a tool for designing content types and components. This documentation gives an overview of the Content-type Builder and covers field options, relations, component usage, and shares data modeling tips.

Types of Contents

- Collection types: content-types that can manage several entries like Blog, Artifacts etc
- Single types: content-types that can only manage one entry. Like Homepage, contact Page
- Components: content structure that can be used in multiple collection types.single types. Although they are technically not proper content-types because they cannot exist independently, components are also created and managed through the Content-type Builder, in the same way as collection and single types.

Create a Content Type

- First we choose above three of one type put a name like home, about and after add a new field and save. As shared or custom component.

- Fields to add as Types also like number, text, media as img,video, date , json , component
- Choose this type of field and give us name and choose like in text as short, long text, media as single, repeating, component as single, repeating.
- After done all click save.

2.3 Content Manager

- After selecting the content type, we save it and go to the **Content Manager** to fill in the data for the Home content type as chosen above.
- It already creates a placeholder for the fields you defined earlier in the **Content Builder**, so just add the data in the following fields.
- After filling in the data, click **Save and Publish** in the top-right corner to publish the content.

3: Set Roles & Permissions on Api

After Content Manager Finised. But first, we need to make sure that the content is publicly accessible through the API:

1. Click on *Settings* at the bottom of the main navigation.
2. Under *Users & Permissions Plugin*, choose *Roles*.
3. Click the Public role.
4. Scroll down under *Permissions*.
5. In the *Permissions* tab, find Your page like(home-page, above)and click on it.
6. Click the checkboxes next to find and findOne.
7. Repeat with *Category*: click the checkboxes next to find and findOne.
8. Finally, click Save.

It will give as endpoint as like **api/home-page** copy this and use.

4.Use Api

After roles done we copy that api endpoint to run with local host like
http://localhost:1338/api/home-page?populate=*
It give us full data as like api json format

Populate:

Use the populate parameter to populate specific fields and the select parameter to return only specific fields with the query results.
Use this populate as single api calls multisections like

http://localhost:1338/api/home-page?populate[footer][populate]=*
This will call only home page footer section only

5. Change the Content

- Before deploy we can local host change the content type builder and contant Manager.
- After deploy the contant Manager only change and publish.

6. WebHook

- Webhooks let Strapi notify external systems when content changes, while omitting the Users type for privacy.
- Webhook used content change trigger on frontend , other url:

Webhooks entry types:

- A new entry is create
- An entry is updated
- An entry is deleted
- A media file is uploaded, etc.

Use of WebHook

Go to Settings → Webhooks
Click Create new webhook

Fill in the Webhook Details

- Name– Any name (for example: “Next.js Rebuild”)

- URL = The endpoint you want Strapi to call (for example, a Next.js API route or a Vercel deploy hook URL)
- Events Select when to trigger the webhook → e.g. Entry Create, Update, Delete
- Enabled Turn it ON

7. Use in Frontent next Js

- Create a nextJs project npx create-next-app@latest frontend
- To fetch content from your Strapi backend, use the Strapi API URL inside your Next.js application.

.